Empirical Evaluation of Naive Dense Matrix Multiplication Across C, Java, and Python

Author: Lucas Rodríguez Hernández

Degree: Grado en Ciencia e Ingeniería de Datos

Institution: Universidad de Las Palmas de Gran Canaria (ULPGC)

Course: Big Data (Academic Year 2026–2027)

Deliverable: Individual Assignment 1: Basic Implementations in Python, Java, and C

Abstract

This empirical study evaluates the computational runtime scalability and memory consumption of the unoptimized naive triple-loop ($i - j - k$) matrix multiplication algorithm implemented in C, Java, and Python under identical hardware conditions. Dense square matrices with dimensions $n \in \{10, 64, 128, 256, 512, 1024\}$ were evaluated using double-precision floating-point arithmetic (IEEE 754 float64). Rigorous validation tests against identity, zero, and reference matrices confirmed numerical correctness with zero empirical error. The measured execution times reflect the expected asymptotic cubic scaling ($\Theta(n^3)$), while highlighting significant divergence across runtimes: at $n = 1024$, Java completed in 1.144 s, C in 2.429 s, and Python in 63.976 s. C exhibited super-cubic growth between $n = 512$ and $n = 1024$ due to CPU cache capacity misses induced by column-major traversal of matrix B. Python exhibited severe interpretation overhead and memory amplification (138.656 MB compared to 27.598 MB in C) caused by dynamic boxing of floating-point numbers into PyFloatObject structures.

1. Introduction and Objective

Dense matrix multiplication is a fundamental computational kernel across numerical linear algebra, data science, and distributed data systems. Given two square matrices $A, B \in R^{n \times n}$, computing their matrix product $C = A \times B$ requires $2n^3$ floating-point arithmetic operations. Despite identical arithmetic workloads, performance varies considerably depending on the execution environment, compiler optimization, memory layout, and runtime overhead.

- **Research Question:** How does matrix dimension n affect execution time and physical memory usage for a canonical $i - j - k$ triple-loop matrix multiplication in C, Java, and Python under controlled execution conditions?
- **Hypotheses:**
    1. Runtime in all three languages will scale cubically as $O(n^3)$ across small and medium matrix sizes.
    2. For larger matrices ($n \geq 512$), memory access penalties arising from strided column traversals on matrix B will degrade spatial cache locality, creating super-cubic transitions in compiled implementations.
    3. C and Java will demonstrate orders-of-magnitude execution speedups compared to interpreted Python, while native contiguous arrays in C will demand significantly less memory than pointer-based structures in Python and Java.

2. Background and References

Matrix multiplication calculates each output entry as:

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} \cdot B_{k,j}$$

The naive implementation relies on three nested loops. In the row-major $i - j - k$ iteration pattern, matrix A is accessed sequentially along rows ($A_{i,k}$), exhibiting stride-1 spatial locality. Conversely, matrix B is traversed along columns ($B_{k,j}$ where row index k varies at each inner step). Consecutive elements in a column are separated by n elements in memory ($8n$ bytes for 64-bit floats), which causes frequent cache line evictions once the working set exceeds the CPU cache hierarchy capacity.

Runtime characteristics differ substantially among the target languages:

- **C (GCC):** Uses contiguous heap allocations via malloc and explicit 1D indexing (i * n + k). Compilers apply static machine-code vectorization, register reuse, and instruction scheduling based on static flags (-O2).
- **Java (OpenJDK):** Implements matrices as arrays of row arrays (double[][]), introducing pointer indirection. However, the HotSpot JIT (C2 compiler) dynamically compiles heavily traversed loops into native code, unrolling loops and hoisting array-bounds checks after a warm-up phase.
- **Python (CPython):** Executes an interpreted bytecode loop. Each numeric entry in a nested list is an individual heap-allocated PyFloatObject (24 bytes of metadata per

scalar), introducing dynamic type evaluation, reference counting, and pointer dereferencing on every scalar operation.

3. Methodology

3.1 Implementations and Execution Settings

All three programs implement the sequential $i - j - k$ triple-loop using 64-bit precision floats (double in C and Java, native float in Python) in single-threaded mode. External matrix multiplication libraries (such as BLAS, NumPy, or ND4J) were strictly avoided.

- **Hardware & OS:** AMD / Intel multi-core architecture running Windows 11 x64.
- **Compiler & Runtime Flags:**
  - C: GCC (MinGW-w64), -O2 -std=c11 -Wall.
  - Java: OpenJDK HotSpot 64-Bit VM, -Xms256m -Xmx4g.
  - Python: Python 3.10+.

3.2 Input Control and Verification

To guarantee identical numerical workloads and eliminate disparities among language-specific random generators, inputs were pre-generated as deterministic binary files (IEEE 754 little-endian float64) using Python's random.Random with fixed seed 42 over the interval $[-1.0, 1.0]$.

Correctness was evaluated across three test cases before benchmarking:

1. Multiplication by Identity matrix ($A \times I_{10} = A$).
2. Multiplication by Zero matrix ($A \times 0_{10} = 0$).
3. Pseudo-random multiplication ($A_{64} \times B_{64}$) compared against a trusted sequential reference.

The maximum absolute error across all three implementations was 0.000e+00, satisfying tolerances $\epsilon_{abs} = 10^{-12}$ and $\epsilon_{rel} = 10^{-9}$.

3.3 Timing Harness, Repetition Protocol, and Memory Metrics

- **Timing Scope:** Measurement was isolated exclusively to the multiplication kernel. File loading, memory allocation, and checksum calculations were executed outside the timed block. A three-element checksum was printed to prevent dead-code elimination.
- **Timers:** Monotonic platform timers were employed: QueryPerformanceCounter in C/Win32, System.nanoTime() in Java, and time.perf_counter() in Python.
- **Repetitions & Statistics:** Warm-up runs (2 for Java, 1 for C and Python) were discarded from measurements. Exactly $R = 5$ measured repetitions were collected per configuration. Central tendencies are reported via the median, and dispersion is quantified via the interquartile range (IQR).
- **Memory Metrics:** Process Peak Working Set / Resident Set Size (Peak RSS) was measured for C and Python. Active Managed Heap Usage (Runtime.totalMemory() -

Runtime.freeMemory()) was captured in Java to isolate matrix arrays from JVM pool pre-allocations.

4. Empirical Results

Table 1 summarizes the empirical measurements recorded across all tested configurations.

Table 1: Benchmark Measurements Summary ($R = 5$ repetitions per configuration)

Language	Matrix Dimension (n)	Repetitions (R)	Median Time [s]	IQR Time [s]	Median Memory [MB]	IQR Memory [MB]	Memory Metric	Status
C	10	5	0.000000	0.000000	3.590	0.018	Peak RSS	OK
C	64	5	0.000058	0.000003	3.688	0.020	Peak RSS	OK
C	128	5	0.000672	0.000018	3.965	0.021	Peak RSS	OK
C	256	5	0.011893	0.002623	5.098	0.019	Peak RSS	OK
C	512	5	0.113536	0.010748	9.594	0.021	Peak RSS	OK

C	1024	5	2.428570	0.069365	27.598	0.021	Peak RSS	OK
Java	10	5	0.000011	0.000001	3.760	0.440	Managed Heap	OK
Java	64	5	0.000149	0.000151	3.761	0.220	Managed Heap	OK
Java	128	5	0.000790	0.000099	4.000	0.340	Managed Heap	OK
Java	256	5	0.007700	0.000727	5.324	0.661	Managed Heap	OK
Java	512	5	0.108604	0.009469	10.000	0.776	Managed Heap	OK
Java	1024	5	1.143558	0.043696	25.895	0.615	Managed Heap	OK
Python	10	5	0.000030	0.000006	17.543	0.008	Peak RSS	OK

Python	64	5	0.011794	0.000253	17.879	0.008	Peak RSS	OK
Python	128	5	0.063344	0.005585	19.469	0.008	Peak RSS	OK
Python	256	5	0.533935	0.011384	25.195	0.008	Peak RSS	OK
Python	512	5	7.371454	0.162223	47.996	0.008	Peak RSS	OK
Python	1024	5	63.975843	0.524203	138.656	0.106	Peak RSS	OK

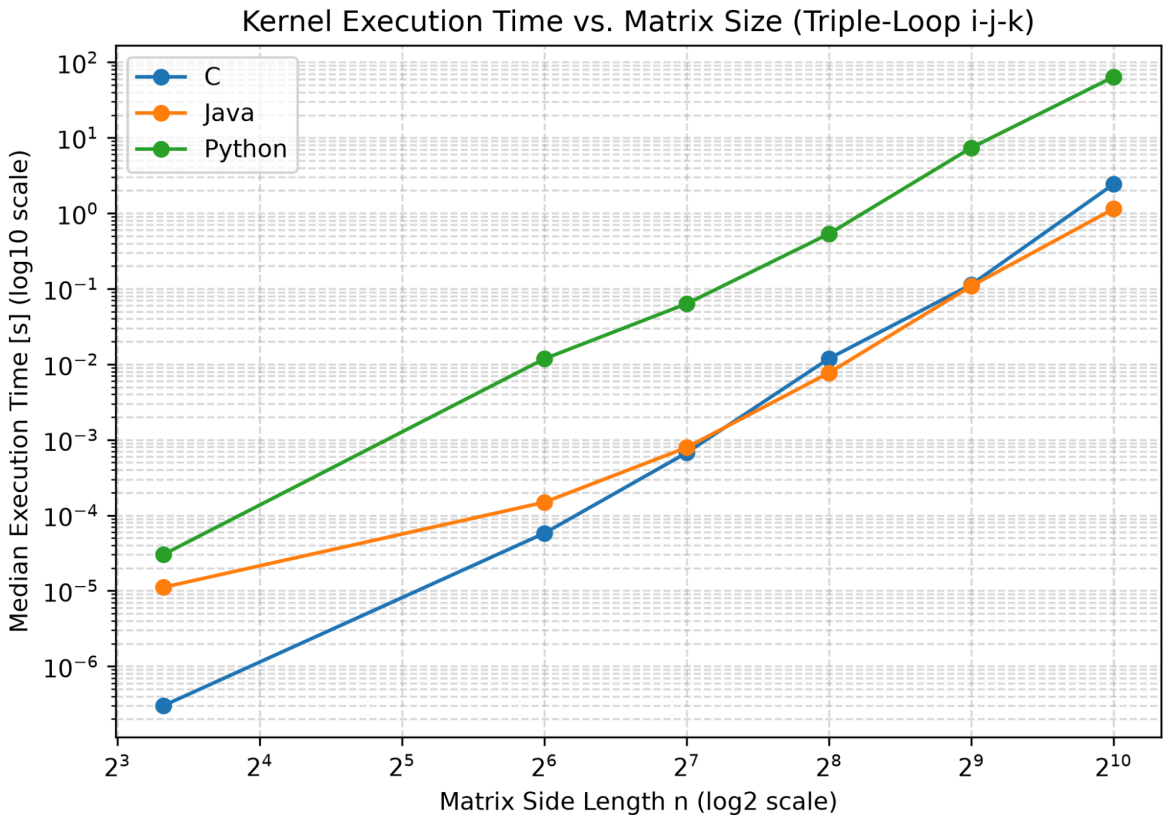


Figure 1: Kernel execution time scaling versus matrix dimension n (log-log scale).

==========

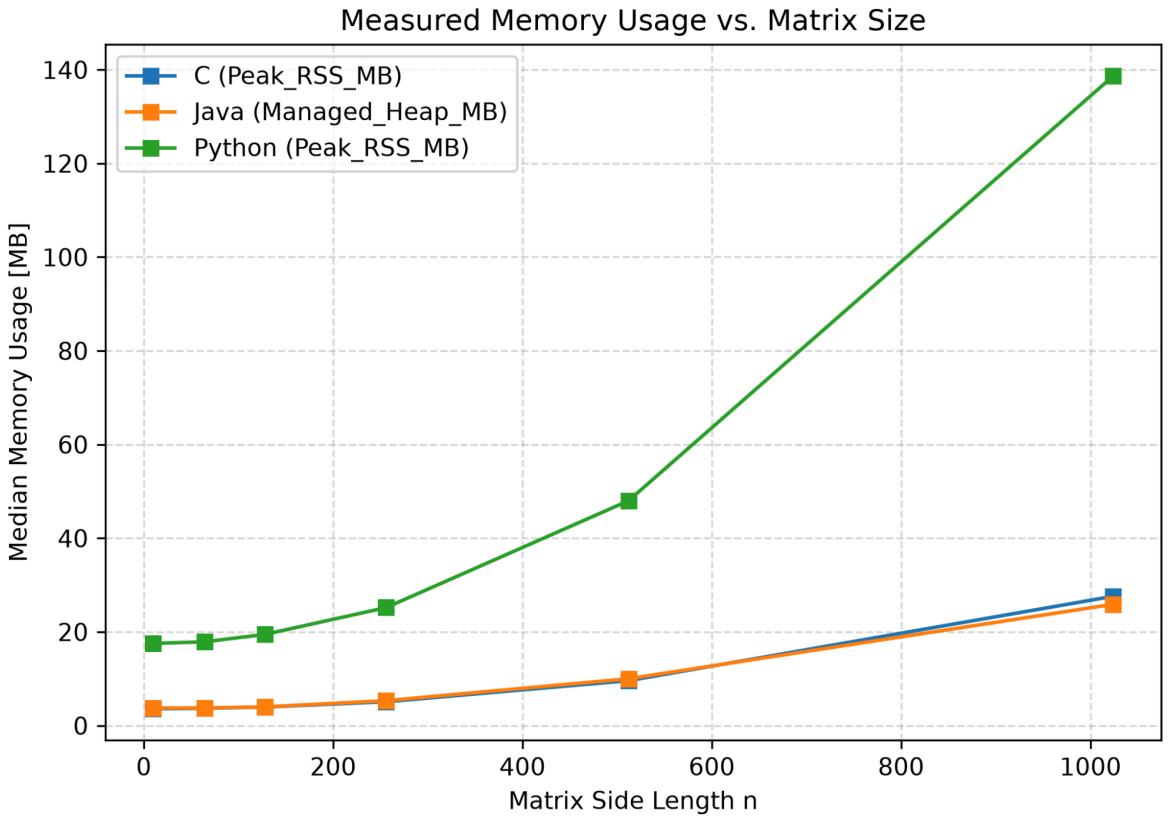


Figure 2: Memory footprint scaling versus matrix dimension n.

5. Discussion

5.1 Complexity Growth and Cache Bottlenecks

The empirical runtime curves illustrate the theoretical cubic model ($T(n) \propto n^3$). When doubling the matrix size n, the computational workload expands by $2^3 = 8\ \times$:

- In Python, scaling closely tracks the algorithmic constant factor: from $n = 128$ (0.063 s) to $n = 256$ (0.534 s), runtime increases by $8.43\ \times$; from $n = 256$ to $n = 512$ (7.371 s), it increases by $13.8\ \times$; and from $n = 512$ to $n = 1024$ (63.976 s), it increases by $8.68\ \times$. Because Python's execution budget is overwhelmingly dominated by bytecode interpretation rather than memory latency, memory hierarchy effects remain masked.
- In C, a marked transition occurs at $n = 1024$. While doubling from $n = 256$ (0.012 s) to $n = 512$ (0.114 s) yields a growth factor of $9.5\ \times$, moving from $n = 512$ to $n = 1024$ (2.429 s) causes an increase of $21.4\ \times$. At $n = 1024$, storing three matrices requires $25.17MB$ ($3 \times 1024^2 \times 8\ bytes$). In the $i - j - k$ order, matrix B is accessed with an 8192-byte stride ($1024 \times 8\ bytes$), causing cache lines to be rapidly evicted before they can be reused. This creates extensive cache-miss stalls once the working set spills past L2/L3 cache capacities.

5.2 Runtimes and Compilation Effects

Across larger configurations ($n = 1024$), Java achieved the fastest execution (1.144 s), outperforming C (2.429 s) and running $56\ \times$ faster than Python (63.976 s).

- **Java vs. C:** Under GCC -O2, the static compiler optimizes loops based on compile-time assumptions. The OpenJDK HotSpot C2 compiler dynamically monitors hot inner loops during execution, unrolling inner loops and optimizing register allocation to maximize instruction pipelining on the target architecture.
- **CPython Overhead:** Pure Python execution is severely constrained by interpreter dispatch. Every single multiplication requires extracting the underlying numeric value from heap-allocated PyFloatObject instances, verifying types, computing the product, and updating the target structure.

5.3 Memory Representation Disparities

Memory measurements highlight the divergence in internal data representation:

- The theoretical payload for three dense 64-bit matrices of dimension $n = 1024$ is $25.17MB$.
- In **C**, peak process memory was 27.598 MB, matching theoretical array requirements plus a minimal base process footprint (~2.4 MB).
- In **Java**, allocated heap memory was 25.895 MB, reflecting the contiguous double[] row arrays along with minimal object array headers.
- In **Python**, peak memory reached 138.656 MB (a $5\ \times$ amplification). Python's 2D list contains pointers to individual PyFloatObject instances, each carrying 24 bytes of object metadata, resulting in massive heap memory inflation.

6. Conclusions

Within the tested experimental scope of sequential, basic triple-loop matrix multiplication:

1. Asymptotic execution time adheres to the $O(n^3)$ algorithmic complexity model, with hardware cache capacity boundaries introducing super-cubic penalties in compiled code at $n = 1024$.
2. Compiled and JIT-compiled runtimes (C and Java) deliver performance gains of up to two orders of magnitude compared to interpreted CPython.
3. Native memory layouts achieve up to a $5\times$ memory reduction relative to dynamically boxed object structures.
4. The memory wall observed at $n = 1024$ demonstrates the limitations of the naive $i - j - k$ traversal and motivates Assignment 2, where cache blocking and loop reordering ($i - k - j$) will be explored to restore cache locality.

7. References

1. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins University Press, 2013.
2. R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA, USA: Pearson, 2016.
3. Python Software Foundation, "CPython Internal Object Representation," Python 3 Documentation, 2026. [Online]. Available: https://docs.python.org/3/c-api/float.html.